ECE 385

Spring 2025

Simple Computer SLC-3.2 in System Verilog
Lab-6

Cristina Natalia Lumban Tobing 3220300398
Artha Amanda Sidauruk 3220300399
TA: Qiyang Wu

Contents

Simple Computer SLC-3.2 in System Verilog 1

1. Introduction 3

1.1. Summarize the basic functionality of the SLC-3 processor 3

2. Written Description and Diagrams of SLC-3 3

2.1. Summary of Operation 3

2.2. Written Description of all .sv Modules 3

Week 1 3

Addition on Week 2 7

2.3. Block Diagram of slc3.sv 8

2.4. Description of the operation of the ISDU (Instruction Sequence Decoder Unit) 9

2.5. State Diagram of ISDU 10

3. Simulations of SLC-3 Instructions 11

4. Post-Lab Questions 13

4.1. Design Resources and Statistics table 13

4.2. Answer of the two questions 13

What is the function of the MEM2IO.sv module? 13

What is the difference between the BR and JMP instructions? 14

What is the purpose of the R signal in Patt and Patel? How do we compensate for the lack of signal in our design? What implications does this have for synchronization? 14

5. Conclusion 14

5.1. Discuss functionality of your design. If parts of your design didn't work, discuss what could be done to fix it 14

5.2. Was there anything ambiguous, incorrect, or unnecessarily difficult in the lab manual or given materials which can be improved for next semester? You can also specify what we did right so it doesn't get changed. 14

1. Introduction

1.1. Summarize the basic functionality of the SLC-3 processor

Collaborating with the Instruction Sequencing and Decode Unit (ISDU), the SLC-3 processor can execute various operations. In this week lab 6, we take SLC-3 to compose of a on-board memory and a processor that can do basic operations such as ADD, AND, XOR, LOAD, STORE, SORT, and a function LOOP

2. Written Description and Diagrams of SLC-3

2.1. Summary of Operation

SLC-3 is 16 bit processor that performs Fetch, Decode, and Execute operations to support 11 instructions, including arithmetic, such as ADD, ADDi, AND, ANDi, NOT; control flow, such as BR, JMP, and JSR; memory access, such as LDR, STR, and PAUSE. Arithmetic instructions work with both registers and immediate values, while control flow instructions manipulate the program counter for branching, jumping, or calling subroutines.

The PAUSE instruction enables user interaction, allowing time to set switches or observe outputs before continuing execution.

2.2. Written Description of all .sv Modules

Week 1

Module: ALU.sv
Inputs: [15:0] A (SR1OUT), [15:0] B (SR2OUT), [1:0] ALUK, SEL
Outputs: [15:0] O_ALU
Description: It performs basic arithmetic and logic operations on two 16-bit inputs, A and B, based on a 2-bit control signal ALUK. It supports four operations, that are, addition, bitwise AND, bitwise NOT (on A), and pass-through of A.
Purpose: This is used to execute core ALU functions in the SLC-3 datapath, enabling arithmetic and logical computation based on the instruction being decoded.

Module: datapath.sv
Inputs: Clk, Reset, GateMARMUX, GateMDR, GatePC, GateALU, [1:0] PCMUX, [1:0] ALUK, [1:0] ADDR2MUX, LD_LED, LD_MDR, LD_IR, LD_CC,LD_MAR, LD_BEN, LD_REG, LD_PC, SR1MUX, SR2MUX, ADDR1MUX, DRMUX, [15:0] MDR_In, MIO_EN
Outputs: BEN, [11:0] LED, [15:0] IR, [15:0] MAR, [15:0] MDR, [15:0] PC

Description: It forms the core of the SLC-3 processor, handling instruction fetch, decode, and execute using components like the ALU, register file, multiplexers, and internal registers.
Purpose: It processes data and control signals to perform operations such as arithmetic, branching, and memory access, enabling full instruction execution within the CPU.

Module: HexDriver.sv
Inputs: [3:0] In0
Outputs: [3:0] Out0
Description: Manage inputs to be displayed on hex.
Purpose: display data we are interested to the HEX display in FPGA board.

Module: ISDU.sv
Inputs: Clk, Reset, Run, Continue, [3:0] Opcode, IR 5, IR 11, BEN
Outputs: LD_MAR, LD_MDR, LD_IR, LD_BEN, LD_CC, LD_REG, LD_PC, LD_LED, GatePC, GateMDR, GateALU, GateMARMUX, [1:0] PCMUX, DRMUX, SR1MUX, SR2MUX, ADDR1MUX, [1:0] ADDR2MUX, ALUK, Mem_CE, Mem_UB, Mem_LB, Mem_OE, Mem_WE
Description: This is a control module that manages the sequencing and control signals required to execute instructions in the SLC-3 processor, using a finite state machine.
Purpose: It generates the appropriate control signals based on the current instruction and processor state, coordinating the datapath to perform operations like fetching, decoding, executing, branching, and memory access.

Module: lab6_toplevel.sv
Inputs: [15:0] S, Clk, Reset, Run, Continue
Outputs: [11:0] LED, [6:0] HEX0, [6:0] HEX1, [6:0] HEX2, [6:0] HEX3, [6:0] HEX4, [6:0] HEX5, [6:0] HEX6, [6:0] HEX7, CE, UB, LB, OE, WE, [19:0] ADDR
Description: This module is the top-level integration of the SLC-3 processor system, connecting inputs, outputs, SRAM interface signals, and display components.
Purpose: It instantiates the SLC-3 processor and test memory, facilitating simulation or hardware testing by routing control, memory, and display signals between modules and external interfaces.

Module: MEM2IO.sv
Inputs: Clk, Reset, [19:0] ADDR, CE, UB, LB, OE, WE, [15:0] Switches, [15:0] Data_from_CPU, [15:0] Data_from_SRAM
Outputs: [15:0] Data_to_CPU, [15:0] Data_to_SRAM, [3:0] HEX0, [3:0] HEX1, [3:0] HEX2, [3:0] HEX3

Description: This module interfaces the SLC-3 processor with input switches and output HEX displays, handling special memory-mapped I/O behavior at address 0xFFFF.
Purpose: It routes data between the CPU and SRAM or switches, and updates the HEX displays with CPU output, enabling basic I/O operations in the SLC-3 system.

Module: memory_contents.sv
Inputs: (Imported from SLC3_2.sv) [15:0] mem array[0:size-1]
Outputs: (Imported from SLC3_2.sv) BEN, [11:0] LED, [15:0] MAR, [15:0] MDR, [15:0] PC, [15:0] IR
Description: The memory contents in the test memory.
Purpose: used for simulation purpose only, take replace of SDRAM.

Module: Mux_FOR_BUS
Inputs: [3:0] S, [15:0] in0, [15:0] in1, [15:0] in2, [15:0] in3
Outputs: [15:0] OUT
Description: 4-input, 16-bit multiplexer controlled by a 4-bit one-hot select signal.
Purpose: It was used to select one of four 16-bit data lines for bus operations.

Module: Mux4IN_16B
Inputs: [1:0] S, [15:0] in0, [15:0] in1, [15:0] in2, [15:0] in3
Outputs: [15:0] OUT
Description: 4-input, 16-bit multiplexer controlled by a 2-bit binary select signal.
Purpose: It selects one of four 16-bit inputs based on binary selection.

Module: Mux2IN_3B
Inputs: S, [2:0] in0, [2:0] in1
Outputs: [2:0] OUT
Description: 2-input, 3-bit multiplexer controlled by a single-bit select signal.
Purpose: It chooses between two 3-bit values based on a control signal.

Module: Mux2IN_16B
Inputs: S, [15:0] in0, [15:0] in1
Outputs: [15:0] OUT
Description: 2-input, 16-bit multiplexer with a single-bit select.
Purpose: It selects between two 16-bit data inputs for data routing.

Module: reg_file
Inputs: [15:0] INPUT_BUS, Clk, Reset, LD_REG, [2:0] SR1, [2:0] SR2, [2:0] input_DR

Outputs: [15:0] O_SR1, [15:0] O_SR2
Description: This is an 8-register file with 16-bit wide registers. Supports two simultaneous reads and one conditional write.
Purpose: It stores and provides access to general-purpose registers. Allows reading two registers and writing one based on control signals.

Module: register_1B
Inputs: Clk, Reset, Load, Data_In
Outputs: Data_out
Description: 1-bit register with asynchronous reset and load control.
Purpose: Stores a single-bit value; resets to 0 or loads new data when enabled.

Module: register_3B
Inputs: Clk, Reset, Load, [2:0] Data_In
Outputs: [2:0] Data_out
Description: 3-bit register with asynchronous reset and load signal.
Purpose: Stores a 3-bit value; used for small control/state data storage.

Module: register_12B
Inputs: Clk, Reset, Load, [11:0] Data_In
Outputs: [11:0] Data_out
Description: 12-bit register with async reset and controlled loading.
Purpose: Holds 12-bit data; useful for storing addresses or partial data words.

Module: register_16B
Inputs: Clk, Reset, Load, [15:0] Data_In
Outputs: [15:0] Data_out
Description: 16-bit register with asynchronous reset and load.
Purpose: Stores full 16-bit words; typically used in datapaths or buses.

Module: SEXT
Inputs: [15:0] IR
Outputs: [15:0] SEXT5, [15:0] SEXT6, [15:0] SEXT9, [15:0] SECT11
Description: Sign-extension unit for 5, 6, 9, and 11-bit fields extracted from a 16-bit instruction.
Purpose: This extends immediate fields to 16 bits for correct signed arithmetic operations.

Module: slc3

Inputs: [15:0] S, Clk, Reset, Run, Continue
Outputs: [11:0] LED, [6:0] HEX0, [6:0] HEX1, [6:0] HEX2, [6:0] HEX3, [6:0] HEX4, [6:0] HEX5, [6:0] HEX6, [6:0] HEX7, CE, UB, LB, OE, WE, [19:0] ADDR
Description: This module contains all the SLC-3 components, and is exactly SLC-3. It contains datapath module which includes all the small components, MEM2IO module and statemachine. Hex signal is used to display write data.
Purpose: It is used as SLC-3 processor which can process the instructions from memory.

Module: tristate.sv
Inputs: Clk, tristate_output_enable, [N-1:0] Data_write
Outputs: [N-1:0] Data_read
Description: This component works as a buffer which can either read data from SRAM to MEM2I0 or write Data to SRAM, if tristate output enable is not active, it will output high-impedance value.
Purpose: This module can connect both port from MEM2IO and SRAM, and accomplish both read and write operations bidirectionally through inout wire data bus.

Addition on Week 2

Module: datapath.sv
Inputs: Clk, Reset, LD_MAR, LD_MDR, LD_IR, LD_BEN, LD_CC, LD_REG, LD_PC, LD_LED, GatePC, GateMDR, GateALU, GateMARMUX, DRMUX, SR1MUX, SR2MUX, ADDR1MUX, MIO_EN, [15:0] MDR_In, [1:0] ADDR2MUX, [1:0] ALUK, [1:0] PCMUX,
Outputs: BEN, [11:0] LED, [15:0] IR, [15:0] MAR, [15:0] MDR, [15:0] PC
Description: A complete 16-bit processor datapath integrating registers, ALU, PC, IR, NZP condition codes, and control logic.
Purpose: This module executes instructions by coordinating data movement, computation, and control signals across the processor's components.

Module: ISDU.sv
Inputs: Clk, Reset, Run, Continue, [3:0] Opcode, IR 5, IR 11, BEN
Outputs: LD_MAR, LD_MDR, LD_IR, LD_BEN, LD_CC, LD_REG, LD_PC, LD_LED, GatePC, GateMDR, GateALU, GateMARMUX, [1:0] PCMUX, DRMUX, SR1MUX, SR2MUX, ADDR1MUX, [1:0] ADDR2MUX, ALUK, Mem_CE, Mem_UB, Mem_LB, Mem_OE, Mem_WE
Description: This is a finite state machine that manages the sequencing and control of instruction execution in a processor.

Purpose: To generate appropriate control signals and manage state transitions based on the current instruction, enabling the datapath to correctly execute operations such as fetching, decoding, memory access, ALU operations, and branching.

Module: Synchronizers.sv
Inputs: Clk, d
Outputs: q
Description: This module is a digital synchronizer that bring asynchronous external signals (like switches/buttons) into an FPGA clock domain.
Purpose: The sync module provides basic synchronization, while sync_r0 and sync_r1 add reset functionality (to 0 or 1, respectively), ensuring predictable startup states.

2.3. Block Diagram of slc3.sv

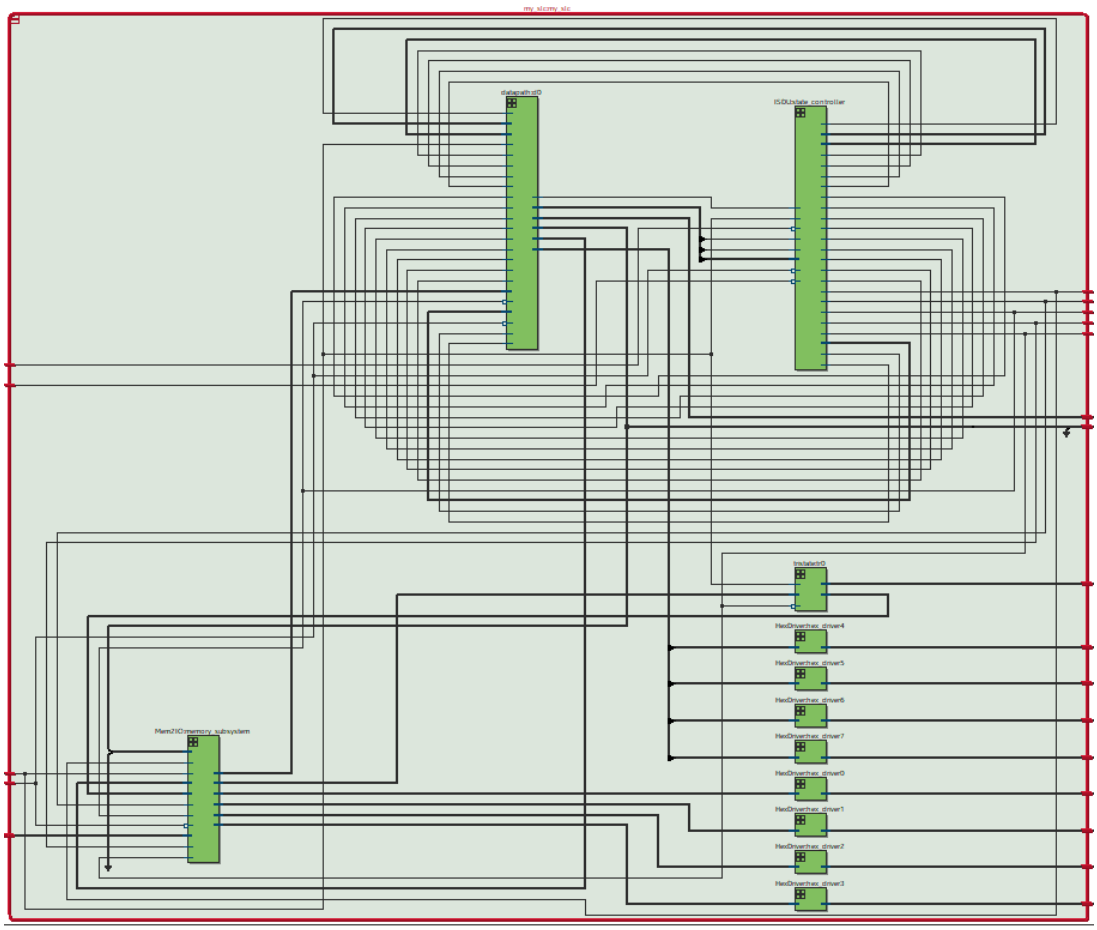

Figure 1: slc3.sv Block Diagram

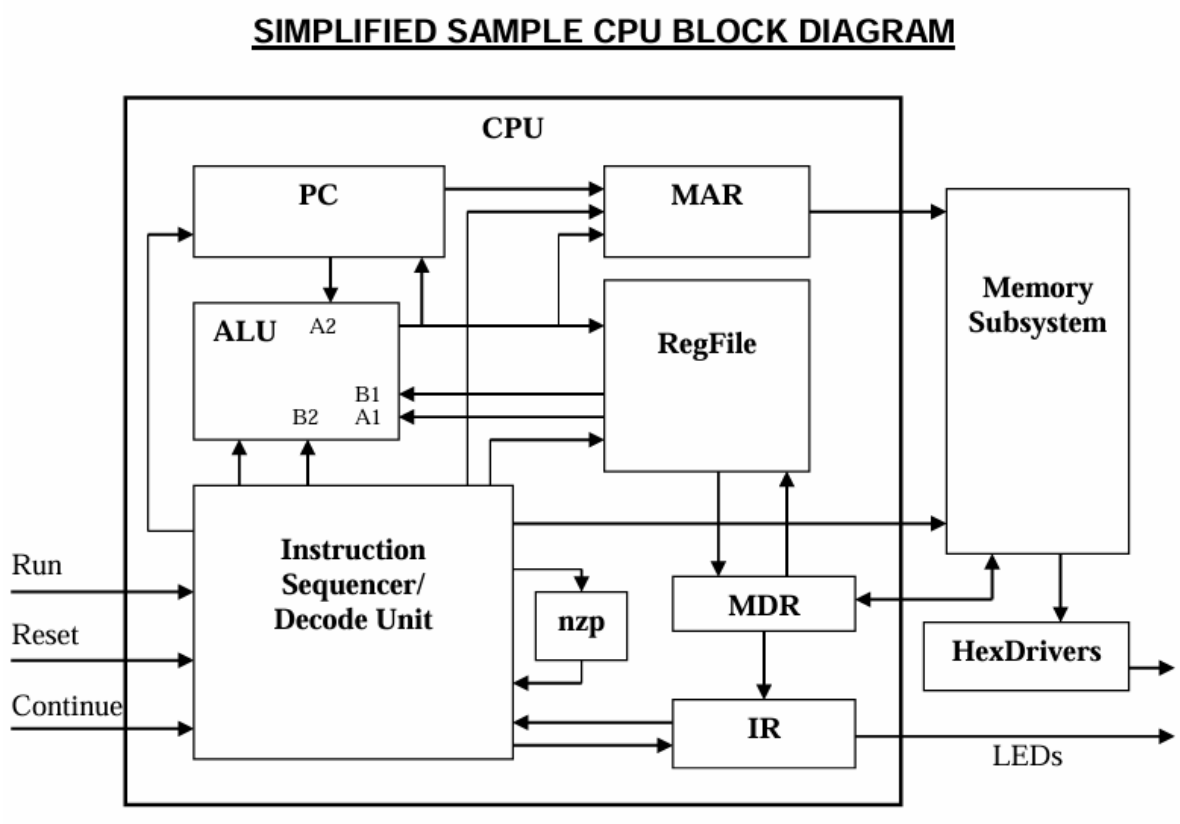

Figure 2: Simplified CPU Block Diagram

2.4. Description of the operation of the ISDU (Instruction Sequence Decoder Unit)

The Instruction Sequence Decoder Unit (ISDU) is a finite state machine (FSM) that controls the Fetch, Decode, and Execute phases of the SLC-3 processor. It responds to inputs like the Run/Continue keys, opcode (IR[15:12]), and IR[5] to generate control signals for registers, ALU operations, memory access, and data flow.

- Halted → Run Transition: When Run is pressed, the ISDU begins execution at state S_18 (Fetch), progressing through memory read (S_33_1-S_33_3) and instruction load (S_35).
- Decode (S_32): The ISDU examines the opcode to branch into the correct execution path (e.g., ADD, AND, BR, JMP, LDR, STR).
- Conditional Execution: For BR (branch), the BEN signal determines if a jump occurs. For JSR, IR[11] selects between PC+offset or BaseR modes.

- Pause Handling: If the opcode is 1101 (PAUSE), the ISDU waits for Continue before resuming.
- Reset: Forces a return to the Halted state, clearing all operations.

The ISDU orchestrates the SLC-3's operation by generating precise control signals (e.g., LD_, *Gate_*, MUX selects, ALUK, memory controls) for each state, ensuring correct data movement, computation, and memory interactions per the current instruction.

2.5. State Diagram of ISDU

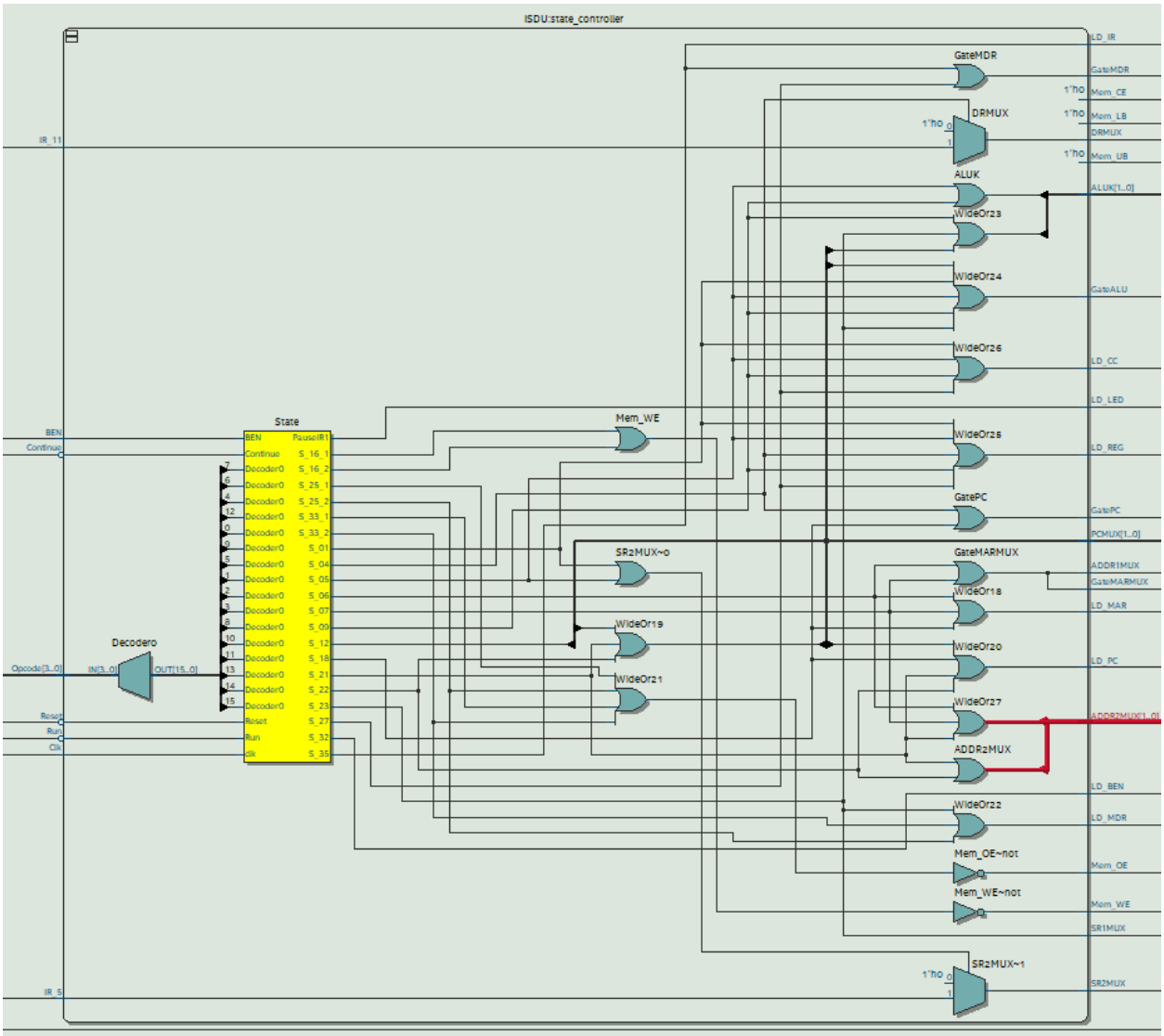

Figure 3: ISDU.sv Block Diagram

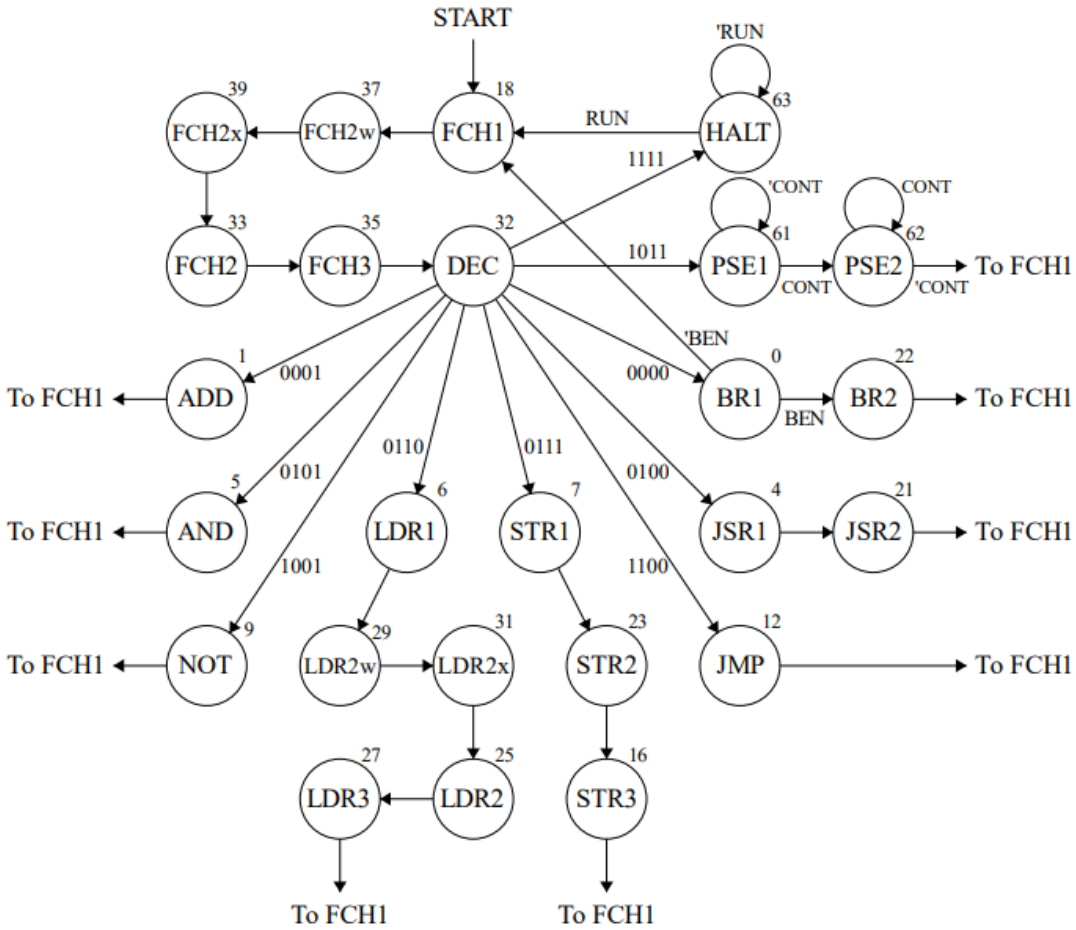

Figure 4: State Diagram of ISDU

3. Simulations of SLC-3 Instructions

LDR (R1, R0, inSW)

This process loads a switch value into register R1 through memory-mapped I/O. First, it fetches an instruction from address x03 to determine the operation, then accesses a special memory location (-1) that's hardwired to the physical switches. Finally, it transfers

the switch value from memory into R2 via the system bus, completing the input operation while maintaining proper data flow through the MAR/MDR/IR registers.

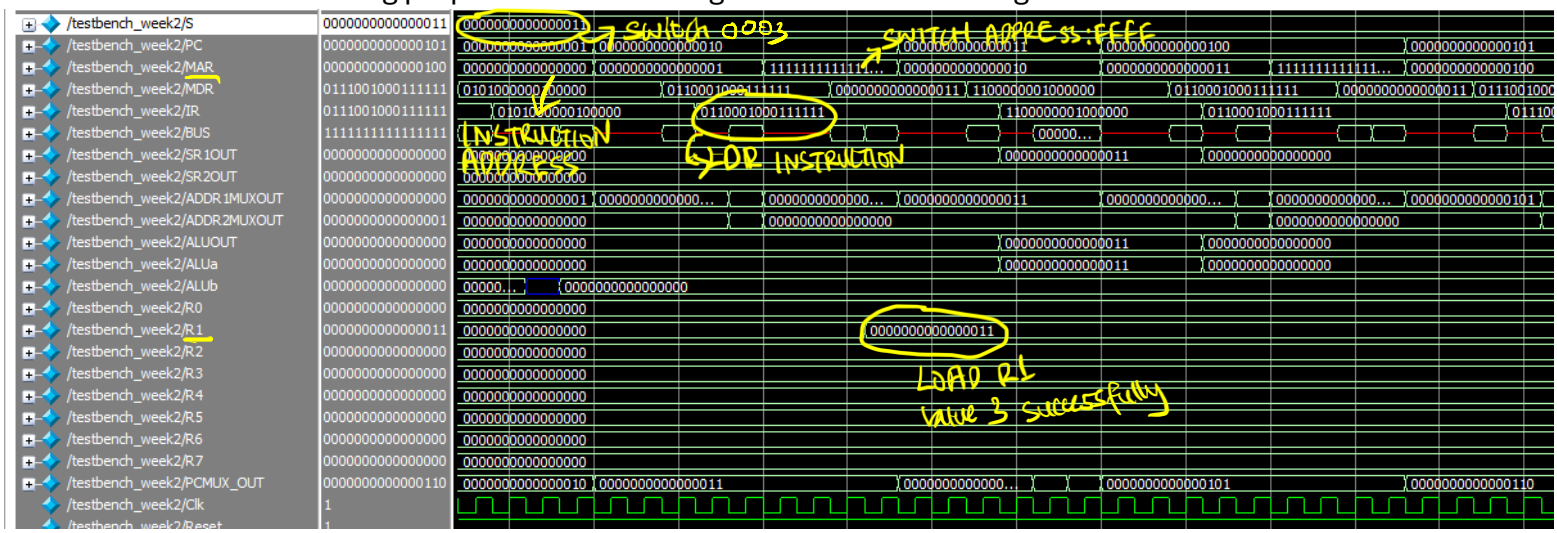

Figure 5: The operation LDR for SLC-3

ADD

Add the value in R1 to itself, and then store the result back in R2

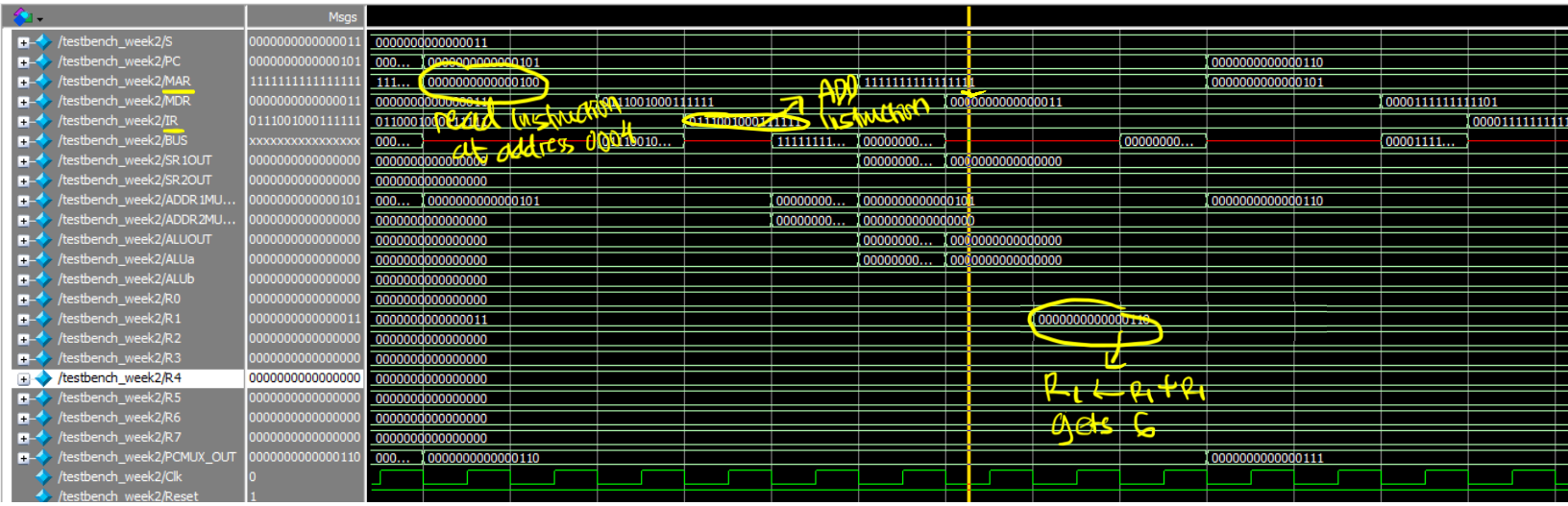

Figure 6: The operation ADD for SLC-3

BR

After storing a value in register R2, the processor automatically updates the condition codes (N, Z, P flags) based on that value. If the stored value is positive (P flag set), the program counter gets decremented by 3, creating a loop that jumps back to the initial instruction at memory address 3. For example, when an addition operation produces the result 6 (which is positive), we can see in the waveform how the program counter rolls back

from address 6 to address 3, effectively restarting that section of the program. This looping behavior continues as long as the operations keep generating positive results in R2.

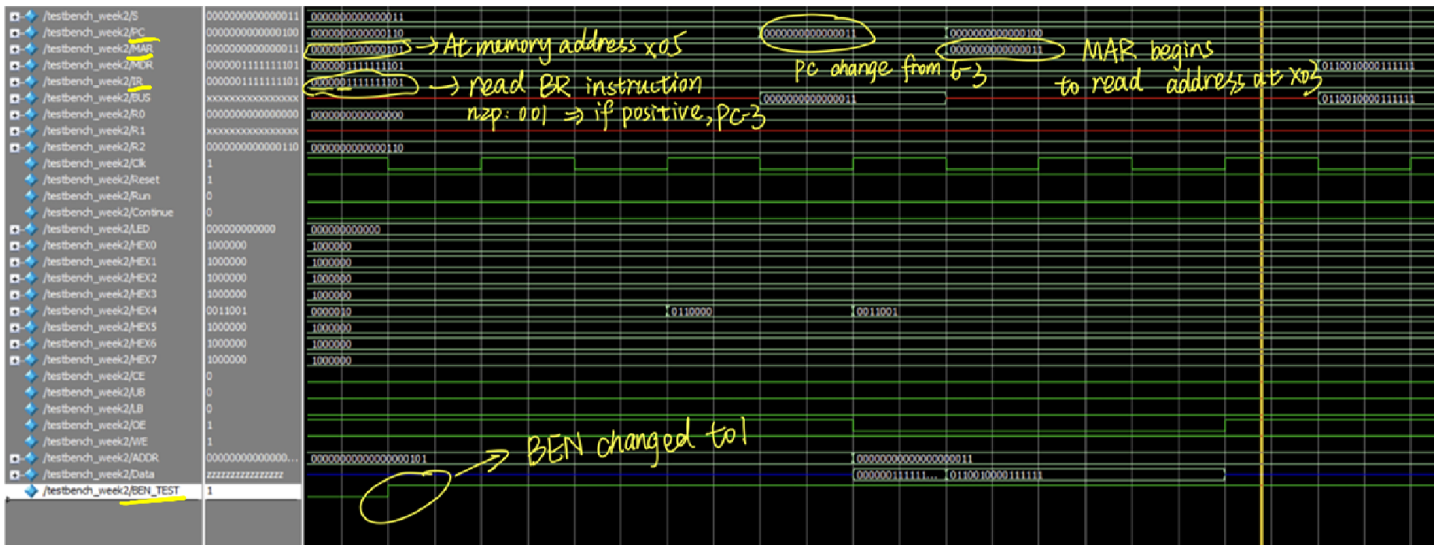

Figure 7: The operation BR for SLC-3

4. Post-Lab Questions

4.1. Design Resources and Statistics table

LUT	735
DSP	0
Memory (BRAM)	0
Flip-Flop	283
Frequency (MHz)	66.51
Static Power (mW)	98.44
Dynamic Power (mW)	7.93
Total Power (mW)	178.48

Table 1: Design Statistics Table

4.2. Answer of the two questions

What is the function of the MEM2IO.sv module?

The MEM2IO.sv module is a bridge between the SLC-3 processor, external SRAM, and I/O devices (switches, HEX displays). It links the processor to memory and physical I/O, enabling memory- mapped I/O operations. Its main functions are:

1. Connects CPU to SRAM – Handles memory reads/writes to external SRAM.
2. Manages I/O devices – Reads switch inputs (e.g., from 0xFFFF) and displays outputs on LEDs/HEX.

3. Data routing – Decides whether CPU accesses memory or I/O (like switches/LEDs).

What is the difference between the BR and JMP instructions?

- BR (Branch):
  - Conditional – Only jumps if status flags (N/Z/P) match the condition in IR[11:9].
  - PC-relative – Jumps to PC + sign-extended offset9 (from IR[8:0]).
- JMP (Jump):
  - Unconditional – Always jumps.
  - Register-based – Jumps to the address stored in a Base Register (BaseR).

What is the purpose of the R signal in Patt and Patel? How do we compensate for the lack of signal in our design? What implications does this have for synchronization?

The R signal in Patt & Patel indicates when memory (SRAM) is ready. Since our design lacks this signal, we add two extra states (S_33_1 and S_33_2) to enforce a fixed 2-cycle delay, ensuring memory stability without needing synchronization. This avoids race conditions while maintaining reliability.

5. Conclusion

5.1. Discuss functionality of your design. If parts of your design didn't work, discuss what could be done to fix it

Our SLC-3 design works correctly—FETCH, MAR, MDR, IR, AND, XOR, and Multiplication operations all function as intended. The FPGA runs smoothly with synchronized button inputs. Overall, the design succeeds, but standard fixes would address common pitfalls.

5.2. Was there anything ambiguous, incorrect, or unnecessarily difficult in the lab manual or given materials which can be improved for next semester? You can also specify what we did right so it doesn't get changed.

The lab has clear instructions and well-structured lab manual that made the design process manageable. The provided code templates and testbenches also saved time and aided debugging. MUX-based design suggestion (replacing tri-state buffers) was helpful. Overall, we think the lab is working pretty well as we follow the instructions from the lab manual.